

KUBERNETES

Finalidad de Kubernetes subir nuestros proyectos web y tareas de administración, etc. Hacer que los dominios (Distintas páginas) apunten a la VPS y con el proxy manager redireccionarlos a puertos distintos y que tengan alta disponibilidad.

La razón por la que se usará MicroK8s es porque si usamos la distribución k3d (versión en desarrollo) No es un kubernetes real ya que si se cae la VPS perdemos directamente el proyecto y los pods. El MicroK8s nos permite gastar menos recursos para poder hacer el proyecto en nuestra VPS.

1 pod → equivale a un Docker compose que realizamos.

En el control plane contiene una etcd (base de datos), una API (basada en NEST, etc), lo que no debe contener bajo ningún concepto en el control plane son Pods ya que eso lo llevará los workers (Nodes).

INSTALACIÓN

Para instalar kubernetes necesitamos 4 máquinas. 1 control plane y 3 nodos, en el que 2 nodos trabajan y la tercera sirve de reserva para que cuando se caiga. (LA PRACTICA GUAPA SERÍA TENER 4 VPS DE CONTABO).

Para la instalación deberemos instalar los paquetes que deberán contener un control plane y los workers unirlos a un cluster.

Con el MicroK8s permite que el control plane sirva también con un worker ahorrando así una máquina (No recomendado, pero es lo que haremos en clase).

PASOS DE LA INSTALACIÓN (MICROK8S)

Actualizamos el sistema

```
jiahaolin@vmi2812274:~$ sudo apt update && sudo apt upgrade -y
```

Desactivamos la swap de linux

```
jiahaolin@vmi2812274:~$ sudo swapoff -a
```

Modificamos una cosa en el fstab, básicamente comentamos la línea para que no se active de nuevo automáticamente

```
sudo sed -i ' / swap / s/^/#/' /etc/fstab
```

```
jiahaolin@vmi2812274:~$ sudo sed -i ' / swap / s/^/#/' /etc/fstab
```

INSTALAMOS MicroK8s

```
Sudo snap install microk8s --classic
```

```
jiahaolin@vmi2812274:~$ sudo snap install microk8s --classic
2026-01-14T11:00:35+01:00 INFO Waiting for automatic snapd restart...
microk8s (1.32/stable) v1.32.9 from Canonical✓ installed
```

Comprobamos si con la instalación se crea el grupo microk8s

```
Cat /etc/group
```

```
docker:x:988:jiahaolin
jiahaolin:x:1000:
jiahaolin:x:1001:
microk8s:x:1002:
```

Añadimos nuestro usuario al grupo microk8s

```
Sudo usermod -aG microk8s $USER
```

```
jiahaolin@vmi2812274:~$ sudo usermod -aG microk8s $USER
```

Comprobamos que se haya añadido

```
microk8s:x:1002:jiahaolin
```

Luego ejecutamos

Newgrp microk8s para poder ejecutar el siguiente comando

```
jiahaolin@vmi2812274:~$ newgrp microk8s
```

Microk8s status --wait-ready

```
jiahaolin@vmi2812274:~$ microk8s status --wait-ready
microk8s is running
high-availability: no
datastore master nodes: 127.0.0.1:19001
datastore standby nodes: none
addons:
  enabled:
    dns                # (core) CoreDNS
    ha-cluster         # (core) Configure high availability on the current node
    helm               # (core) Helm - the package manager for Kubernetes
    helm3              # (core) Helm 3 - the package manager for Kubernetes
```

Habilitamos estos servicios

microk8s enable dns ingress storage metrics-server

```
jiahaolin@vmi2812274:~$ microk8s enable dns ingress storage metrics-server

Enabling metrics-server
serviceaccount/metrics-server created
clusterrole.rbac.authorization.k8s.io/system:aggregated-metrics-reader created
clusterrole.rbac.authorization.k8s.io/system:metrics-server created
rolebinding.rbac.authorization.k8s.io/metrics-server-auth-reader created
clusterrolebinding.rbac.authorization.k8s.io/metrics-server:system:auth-delegator created
clusterrolebinding.rbac.authorization.k8s.io/system:metrics-server created
service/metrics-server created
deployment.apps/metrics-server created
apiservice.apiregistration.k8s.io/v1beta1.metrics.k8s.io created
clusterrolebinding.rbac.authorization.k8s.io/microk8s-admin created
Metrics-Server is enabled
```

Y vemos que se nos haya generado unos pods con el siguiente comando.

microk8s kubectl get pods -A

```
Metrics-Server is enabled
jiahaolin@vmi2812274:~$ microk8s kubectl get pods -A
NAMESPACE   NAME                                                    READY   STATUS    RESTARTS   AGE
ingress      nginx-ingress-microk8s-controller-5zcd9              1/1     Running   0          63s
kube-system  calico-kube-controllers-5947598c79-mtcrq             1/1     Running   0          47m
kube-system  calico-node-szsb2                                     1/1     Running   0          47m
kube-system  coredns-79b94494c7-wp62p                             1/1     Running   0          47m
kube-system  hostpath-provisioner-c778b7559-dh54s                 1/1     Running   0          60s
kube-system  metrics-server-7dbd8b5cc9-k28v1                     1/1     Running   0          57s
```

Ahora instalamos el kubectl en nuestra VPS, no queremos usar el de microk8s, ya que eso nos limitaría solamente a poder conectarnos a esa kubernetes. Al instalarlo de manera global podemos conectarnos a otras kubernetes.

Sudo snap install kubectl --classic

```
jiahaolin@vmi2812274:~$ sudo snap install kubectl --classic
kubectl 1.34.3 from Canonical✓ installed
```

```
jiahaolin@vmi2812274:~$ ls -la
. .bash_history .bashrc .docker .gitconfig .lessht .profile .sudo_as_admin_successful .wget-hsts examen jiahaohlc megacrossover
.. .bash_logout .cache .dotnet .kube .local .ssh .vscode-server devops gitremote kubernetes snap
jiahaolin@vmi2812274:~$ cd .kube
jiahaolin@vmi2812274:~/.kube$ ls
cache
jiahaolin@vmi2812274:~/.kube$
```

Ahora creamos un fichero config para cada kubernetes que tengamos que en este caso es solamente 1.

Microk8s config > ~/.kube/config

```
jiahaolin@vmi2812274:~/.kube$ microk8s config > ~/.kube/config
```

```
jiahaolin@vmi2812274:~/.kube$ ls
cache config
```

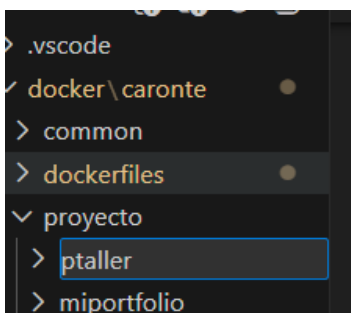
Este archivo contiene el certificado, nombre del servidor, el servidor, etc. básicamente toda la configuración de un kubernetes que en este caso nos pasa el clúster del microk8s.

Podemos observar un nodo que se ha creado.

kubectl get nodes -o wide

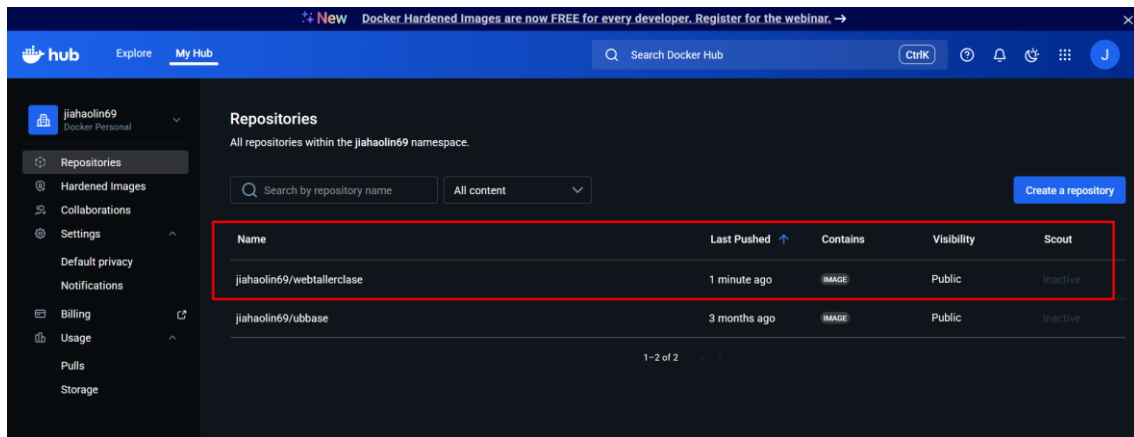
```
jiahaolin@vmi2812274:~/.kube$ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
vmi2812274    Ready     <none>    69m   v1.32.9   194.163.147.140 <none>        Ubuntu 24.04.3 LTS   6.8.0-85-generic containerd://1.6.36
```

Creamos un proyecto lo llamamos “ptaller” en nuestro proyecto de Docker directamente



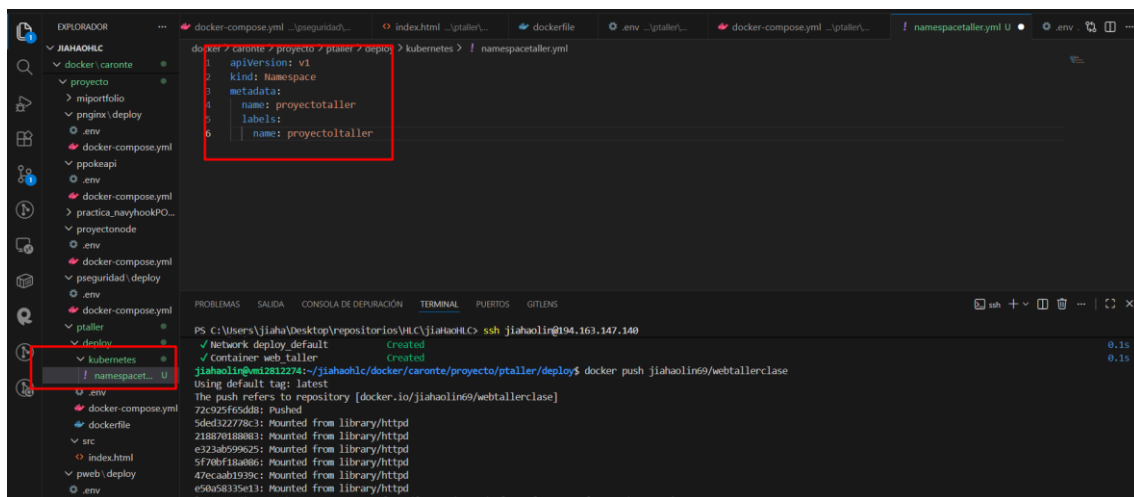
Creamos todo y montamos la imagen y lo subimos al Docker hub

```
✓ jiahaolin69/webtallerclase Built
jiahaolin@vmi2812274:~/jiahaohlc/docker/caronte/proyecto/ptaller/deploy$ docker push jiahaolin69/webtallerclase
Using default tag: latest
The push refers to repository [docker.io/jiahaolin69/webtallerclase]
17ccc3836282: Pushed
5ded322778c3: Mounted from library/httpd
218870188083: Mounted from library/httpd
e323ab599625: Mounted from library/httpd
5f70bf18a086: Mounted from library/httpd
47ecaab1939c: Mounted from library/httpd
e50a58335e13: Mounted from library/httpd
latest: digest: sha256:2a84b07209929cbbf8ec182ea8d246b8edbe72039674ff3995acb080187ebd02 size: 1779
jiahaolin@vmi2812274:~/jiahaohlc/docker/caronte/proyecto/ptaller/deploy$
```



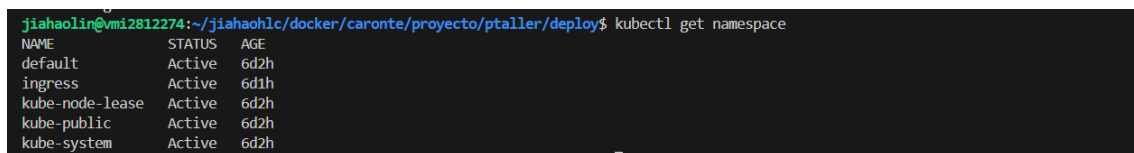
Comenzamos con kubernetes DE HECHO KUBERNETE COMIENZA CUANDO SUBIMOS LA IMAGEN A DOCKER HUB

Para ello primero creamos nuestro namespace.yml

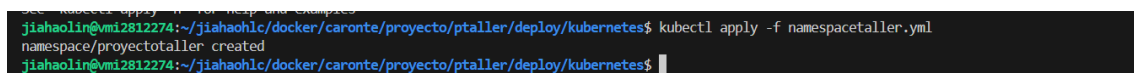


Para recibir los namespaces que hay ejecutamos el comando

Kubectl get namespace



Lo subimos a la VPS y ejecutamos el comando



Y si volvemos a ejecutar el comando saldrá el namespace

```

namespace/projectotaller created
jiahao1in@vml2812274:~/jiahao1c/docker/caronte/proyecto/ptaller/deploy/kubernetes$ kubectl get namespace
NAME                STATUS   AGE
default             Active   6d2h
ingress             Active   6d1h
kube-node-lease     Active   6d2h
kube-public         Active   6d2h
kube-system         Active   6d2h
projectotaller      Active   38s

```

Ahora creamos el POD

```

docker > caronte > proyecto > ptaller > deploy > kubernetes > ! deploywebtaller.yml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: webtaller-deployment
5    namespace: projectotaller
6  spec:
7    replicas: 3
8    selector:
9      matchLabels:
10       app: webtaller
11  template:
12    metadata:
13      labels:
14       app: webtaller
15    spec:
16      containers:
17      - name: webtaller
18        image: jiahao1in69/webtallerclase
19        ports:
20        - containerPort: 80

```

Lo subimos y lo aplicamos

```

jiahao1in@vml2812274:~/jiahao1c/docker/caronte/proyecto/ptaller/deploy/kubernetes$ kubectl apply -f deploywebtaller.yml
deployment.apps/webtaller-deployment created

```

Y se han creado los deploy y pods

```

jiahao1in@vml2812274:~/jiahao1c/docker/caronte/proyecto/ptaller/deploy/kubernetes$ kubectl get deploy -n projectotaller
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
webtaller-deployment 3/3     3             3           16s

```

Ahora hacemos el servicio

```

1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: webtaller-service
5    namespace: projectotaller
6  spec:
7    type: NodePort
8    selector:
9      app: webtaller
10   ports:
11   - port: 80
12     targetPort: 80
13     nodePort: 30080

```

Una vez aplicado el deploy ya tenemos creado un kubernetes con 3 replicas que nos permite ver nuestra pagina desde el puerto 30080

```

← → ↻ No seguro http://194.163.147.140:30080

```

Microk8s

Prueba de Microk8s

Ahora vamos a crear una especie de proxy con el ingress

```
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: webtaller-ingress
5    namespace: proyectotaller
6    annotations:
7      nginx.ingress.kubernetes.io/rewrite-target: /
8  spec:
9    rules:
10     - host: webtaller.jiahao.com
11       http:
12         paths:
13           - path: /
14             pathType: Prefix
15             backend:
16               service:
17                 name: webtaller-service
18                 port:
19                   number: 80
```

Y lo aplicamos

```
Already up to date.
jiahao@lin-w12812274:~/jiahao/h1/docker/caronte/proyecto/ptaller/deploy/kubernetes$ kubectl apply -f ingresswebtaller.yml
ingress.networking.k8s.io/webtaller-ingress created
jiahao@lin-w12812274:~/jiahao/h1/docker/caronte/proyecto/ptaller/deploy/kubernetes$ kubectl get ingress webtaller-ingress -n proyectotaller
NAME          CLASS    HOSTS                ADDRESS      PORTS     AGE
webtaller-ingress public   webtaller.jiahao.com 127.0.0.1    80        81s
```

Ahora entro a Arsys (mi dominio) y en entradas DNS me añado esto.

arsys

Mis Productos

Mis Facturas

Mis Datos

941 620 100

Hola, bienvenido JIA HAO LIN WU (ID: 645932)

ES

Cerrar sesión

Tienda

Eliminar

☒ Entrada DNS

Tipo

Valor

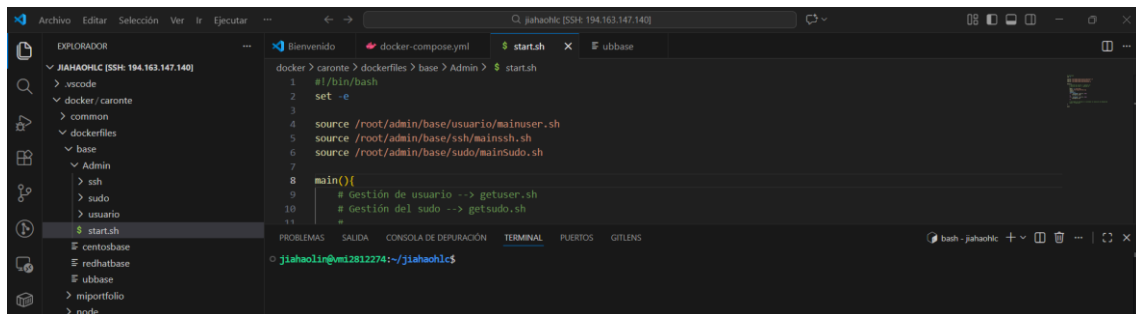
Modificar

☐	1761846866523...domainkey.megacrossover.es	TXT	v=DQM1: k=rsz: p=MIBIJANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBcGkCAQEAps90WVOZ2H2KP3ZeZ0d06.hRTH48GPTt4567JcNl-Sfk vQA73atuTzEBreMJZY1pYhN3XQRFXR1r1ejuH1aOMAHyVvZzbr7aaBeFC0nqTAJltcWdY8T8UIMR2NRu3p67hdj8 9Y5npd2qL-rV9yHUCMUT/HZ0z14-V0/ CjAdhkwAA8BH0duZEy3lr-QrRLWaoYRYUAlUEhJA0qg4Y2NvqgBfBn3d92MULko0BfV5Q5MQJDgyhuJukAJDQ9Hhff 8MWSl3i+g7NxDd/9H4VFKUzyQk48L5fa9ljugpzZ8l6jmlYromKIGA1K2z3VklzROZYDWUWMMYmwCM2SEzQIDA QAB	
☐	autoconfig.megacrossover.es	CNAME	autoconfig.serviciodecorreo.es	
☐	autodiscover.megacrossover.es	CNAME	autodiscover.serviciodecorreo.es	
☐	control.megacrossover.es	CNAME	pdv.servidoresdns.net	
☐	megacrossover.es	SPF	v=spf1 include:_spf.serviciodecorreo.es ~all	
☐	megacrossover.es	A	194.163.147.140	
☐	megacrossover.es	MX 10	mx.serviciodecorreo.es	
☐	webmail.megacrossover.es	CNAME	serviciodecorreo.es	
☐	www.megacrossover.es	A	194.163.147.140	

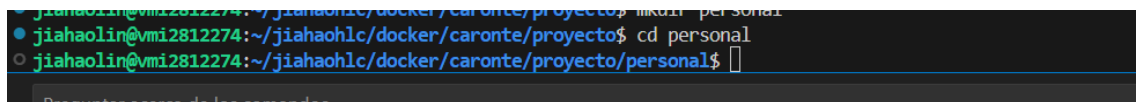
HELM

Es un packet manager de Kubernetes

Ahora vamos a crear una automatización con helm hacemos conexión remota a la VPS ya que vamos a trabajar directamente desde una VPS

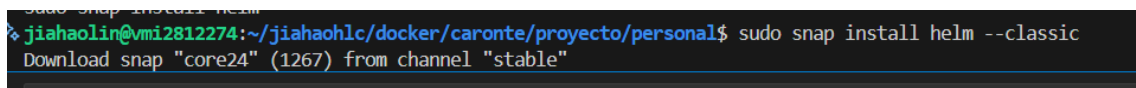


Entramos en proyecto y creamos personal

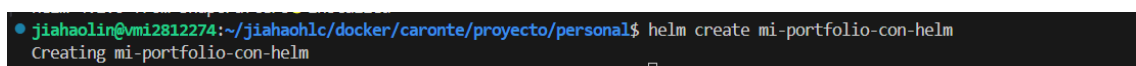


Y ponemos el siguiente comando

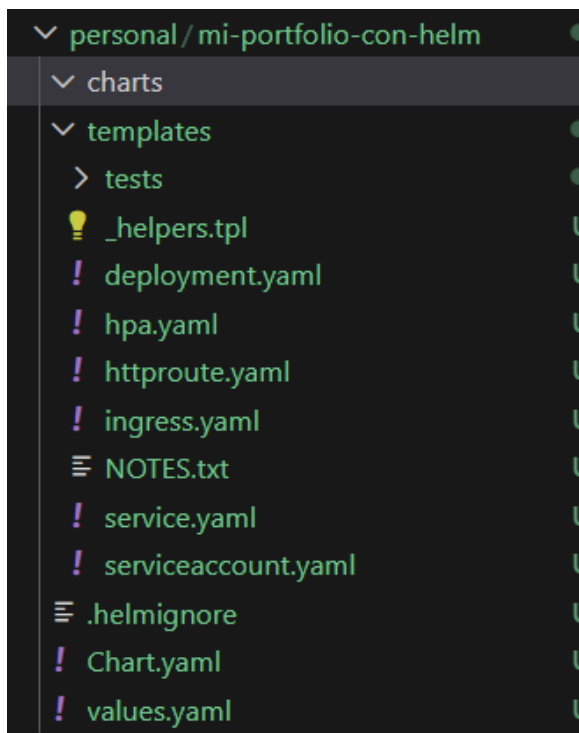
Para instalar Helm



Y creamos un proyecto con portfolio

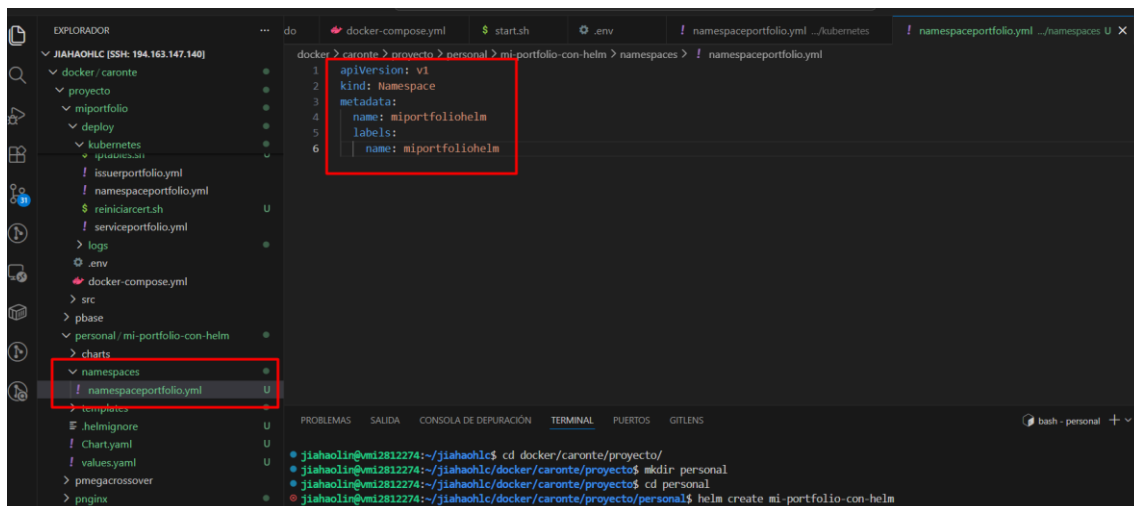


Nos crea este listado de directorio

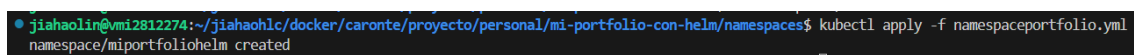


Estos servicios controlan nuestras aplicaciones.

Ahora lo que haremos es crear un namespaces.



Y lo aplicamos



Y en los archivos cambiamos el namespace a lo que hemos creado

En deploy

```
docker > caronte > proyecto > personal > mi-portfolio-con-helm > templates > ! deployportfolio.yml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: webportfolio-deployment
5    namespace: miportfoliohelm
6  spec:
7    replicas: 3
8    selector:
9      matchLabels:
10       app: webportfolio
11    template:
12      metadata:
13        labels:
14          app: webportfolio
15      spec:
16        containers:
17          - name: webportfolio
18            image: jiahaolin69/miportfolio
19            ports:
```

En ingress

```
docker > caronte > proyecto > personal > mi-portfolio-con-helm > templates > ! ingressportfolio.yml
1  apiVersion: networking.k8s.io/v1
2  kind: Ingress
3  metadata:
4    name: webtaller-ingress
5    namespace: miportfoliohelm
6  annotations:
7    nginx.ingress.kubernetes.io/rewrite-target: /
8  spec:
9    rules:
10     - host: portfolio.megacrossover.es
11       http:
12         paths:
13           - path: /
14             pathType: Prefix
15             backend:
16               service:
17                 name: portfolio-taller
18                 port:
19                   number: 80
```

Y en Service

```

docker > caronte > proyecto > personal > mi-portfolio-con-helm > templates > ! ser
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: portfolio-taller
5    namespace: miportfoliohelm
6  spec:
7    type: NodePort
8    selector:
9      app: webportfolio
10   ports:
11     - port: 80
12       targetPort: 80
13       nodePort: 30081

```

Y borramos lo anterior con

kubectl delete namespace miportfolio

```

jiahaojin@vmi2812274:~/jiahaojin/docker/caronte/proyecto/personal/mi-portfolio-con-helm/namespaces$ kubectl delete namespace proyectoportfoli
namespace "proyectoportfolio" deleted
[]

```

Ahora actualizamos el Helm con el siguiente comando

Helm upgrade --install [nombre del proyecto] . (indica el directorio) -n [namespace del proyecto] --create-namespace (no lo usamos nunca porque lo hemos creado en un fichero para tenerlo registrado)

```

projectcaronte: ~$ helm upgrade --install mi-portfolio-con-helm . -n miportfoliohelm
Release "mi-portfolio-con-helm" does not exist. Installing it now.
NAME: mi-portfolio-con-helm
LAST DEPLOYED: Tue Jan 27 13:21:42 2026
NAMESPACE: miportfoliohelm
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
NOTES:
1. Get the application URL by running these commands:
  export POD_NAME=$(kubectl get pods --namespace miportfoliohelm -l "app.kubernetes.io/name=mi-portfolio-con-helm,app.kubernetes.io/instance=mi-portfolio-con-helm" -o jsonpath="{.items[0].metadata.name}")
  export CONTAINER_PORT=$(kubectl get pod --namespace miportfoliohelm $POD_NAME -o jsonpath="{.spec.containers[0].ports[0].containerPort}")
  echo "Visit http://127.0.0.1:8080 to use your application"
  kubectl --namespace miportfoliohelm port-forward $POD_NAME 8080:$CONTAINER_PORT

```

Si ejecutamos con `kubectl get all -n miportfoliohelm`

```
● jiahaolin@vm12812274:~/jiahaohic/docker/caronte/proyecto/personal/mi-portfolio-con-helm$ kubectl get all -n miportfoliohelm
NAME                                READY   STATUS    RESTARTS   AGE
pod/mi-portfolio-con-helm-5654448b6-6sfmv   1/1     Running   0          6m26s
pod/webportfolio-deployment-9cb45fd7-52l8s   1/1     Running   0          6m26s
pod/webportfolio-deployment-9cb45fd7-bmjzn   1/1     Running   0          6m26s
pod/webportfolio-deployment-9cb45fd7-wpq5l   1/1     Running   0          6m26s

NAME                                TYPE               CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
service/mi-portfolio-con-helm        ClusterIP          10.152.183.188 <none>        80/TCP           6m26s
service/portfolio-taller              NodePort           10.152.183.163 <none>        80:30081/TCP     6m26s

NAME                                READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/mi-portfolio-con-helm  1/1     1             1           6m26s
deployment.apps/webportfolio-deployment 3/3     3             3           6m26s

NAME                                DESIRED   CURRENT   READY   AGE
replicaset.apps/mi-portfolio-con-helm-5654448b6  1         1         1       6m26s
replicaset.apps/webportfolio-deployment-9cb45fd7  3         3         3       6m26s
```

Y podemos observar que todo está corriendo

Si da error tenemos que hacer un `kubectl log` o describe con el nombre del POD y podemos ver el error que nos falla

VARIABLES DE ENTORNO (configmap)

Con los variables vamos a poder reutilizar la configuración en distintos proyectos ya que solamente será cambiar el fichero del variable de entorno

Primero vamos con el namespace en los 3 templates

```
personal > mi-portfolio-con-helm > templates > ! deployportfolio.yml
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: webportfolio-deployment
5    namespace: {{ .Release.Namespace }}
6  spec:
7    replicas: 3
8    selector:
9      matchLabels:
10       app: webportfolio
11    template:
12      metadata:
13        labels:
14         app: webportfolio
15      spec:
16        containers:
17         - name: webportfolio
18           image: jiahaolin69/miportfolio
19           ports:
20             - containerPort: 80
```